

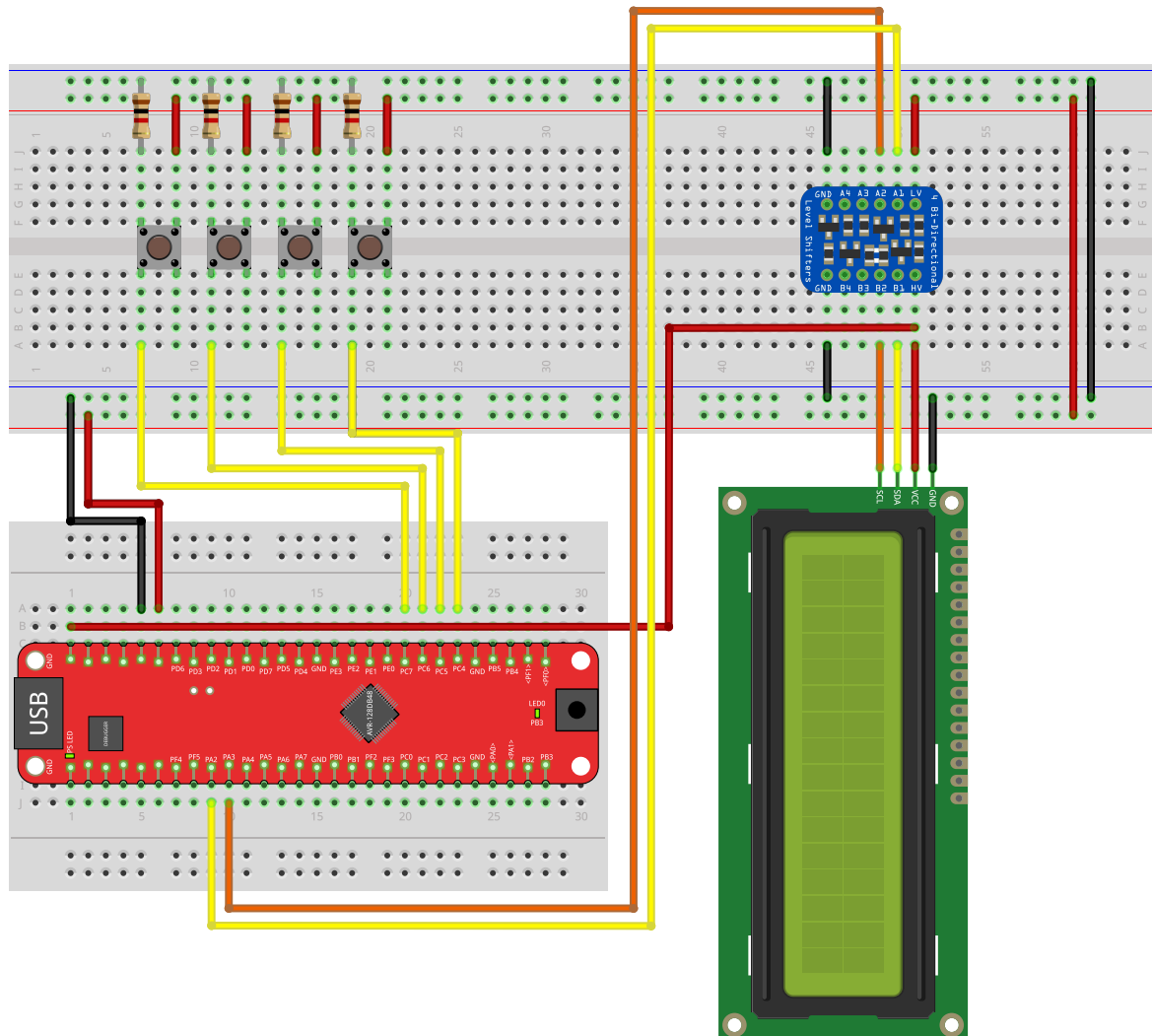
Ansteuerung eines *Liquid Crystal Displays* (LCDs)

Übersicht

<i>Hello Display</i>	5
<i>Count Up</i>	5
<i>Ping Pong</i>	6
<i>Binary Calculator II</i>	6
Literaturverzeichnis	8

In dieser Übung werden in dem Versuchsaufbau die LEDs durch ein Liquid Crystal Display (LCD) vom Typ HD44780 ersetzt (Siehe [Versuchsaufbau 3](#), welches dem Lösen der weiteren Praktikumsaufgaben dient. Dieses Anzeigemodul für 16 x 2 Zeichen wurde durch einen I²C I/O-Expander erweitert, welcher nun wie bei dem [LCD-Backboard](#) zu sehen angeschlossen wird.

Das Displaymodul ist wie folgt anzuschließen:



fritzing

Abbildung 1: Ein Liquid Crystal Display und vier Buttons. Das TXB0104-Bauteil stammt aus der Adafruit Fritzing Library [1]

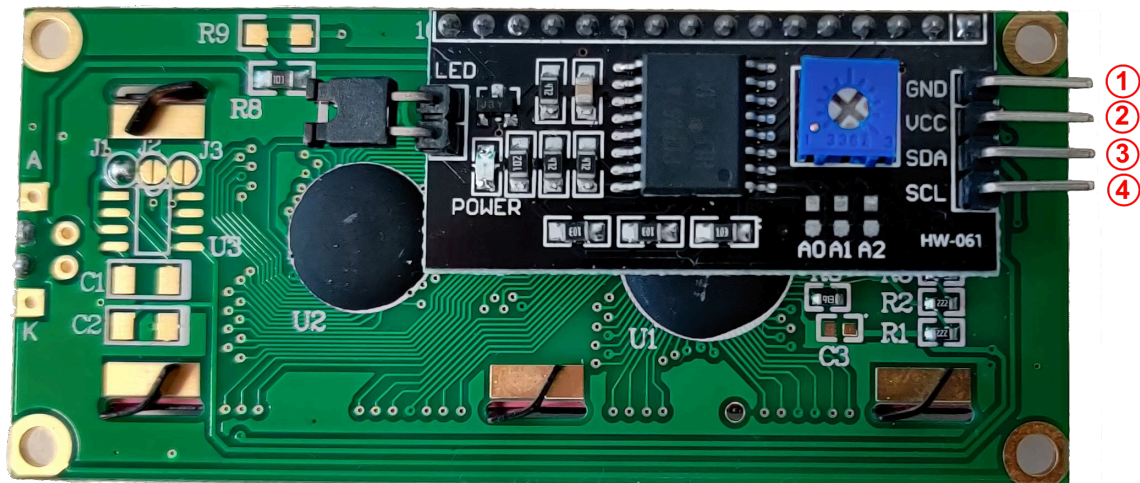


Abbildung 2: Pinbelegung

Nummer	Bezeichnung	Pin	Beschreibung
1	GND	GND	Ground
2	VCC	5V	Versorgungsspannung
3	SDA	PA2	Serial Data
4	SCL	PA3	Serial Clock

Tabelle 1: Pinbelegung

Um das LCD ansteuern zu können, werden Sie die zur Verfügung gestellten Bibliotheken *I2C_LCD* sowie *AVR128DB48_I2C* nutzen.

Die zugehörigen Dateien befinden sich im Ordner „Includes“ (Zu finden auf Moodle). Diesen müssen Sie in ihr Projekt importieren. In Microchip Studio erstellen Sie dazu im Solution-Explorer mittels „Add new folder“ den Ordner „Includes“ - am einfachsten auf der selben Ebene wie ihr Hauptprogramm - und dessen Unterordner „AVR128DB48_I2C“ sowie „I2C_LCD“. Fügen Sie die heruntergeladenen Dateien entsprechend ihrem Verzeichnis mittels „Add existing item“ ihrem Projekt hinzu.

Deren Pfade fügen Sie nun ihren Projekteinstellungen hinzu. Dazu navigieren Sie zu:

Project → Properties → Toolchain → AVR/GNU Compiler → Directories

und fügen hier über den Button mit dem grünen Plus die Pfade „Includes/AVR128DB48_I2C“ und „Includes/I2C_LCD“ innerhalb Ihres Projektes hinzu.

Inkludieren Sie die Datei „I2C_LCD.h“ in ihr Programm.

In Visual Studio Code erstellen Sie ebenfalls einen neuen Ordner namens „Includes“ mit den beiden Unterordnern „AVR128DB48_I2C“ sowie „I2C_LCD“ und legen die Dateien in den jeweiligen Ordnern ab. Dann müssen die beiden Dateien AVR128DB48_I2C.c und I2C_LCD.c dem Projekt hinzugefügt werden, das kann wie folgt getan werden:

STRG+SHIFT+P → MPLAB: Edit Project Properties (Ui) → Fileset

Hier können dann in der Tabelle unter „Include“ die Dateien hinzugefügt werden.

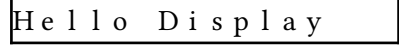
Damit die Bibliotheken richtig hinzugefügt werden, müssen die beiden Ordner noch als Library-Pfad hinzugefügt werden.

STRG+SHIFT+P → MPLAB: Edit Project Properties (Ui) → C Compiler → Include directories

Hier müssen die Pfade zu den beiden Ordnern angegeben werden.

Hello Display

Nutzen Sie die Bibliothek, um auf der **unteren** Zeile des Displays den Text *Hello Display* auszugeben.



He l l o D i s p l a y

Hierzu müssen Sie sich zunächst die Dateien *I2C_LCD.h* und *I2C_LCD.c* genauer anschauen und verstehen, welche Möglichkeiten Ihnen mit dieser Bibliothek zur Verfügung stehen.

- Wie können Sie einen String auf das Display schreiben?
- Wie können Sie in die zweite Zeile des Displays schreiben?
- Auf welchem Weg können Sie den Text überschreiben?

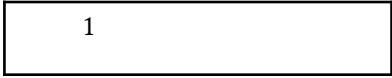
Count Up

Schreiben Sie ein Programm, welches in einer über den Code einstellbaren Geschwindigkeit bis 1.000 zählt. Die Zahlen sollen während des Zählens einzeln auf dem Display ausgegeben werden. Überlegen Sie sich hierzu insbesondere, wie Sie einen Zahlen-Datentypen (z.B. *int32_t*) in eine Zeichenkette umwandeln können, **ohne** bereits vorgefertigte Funktionen dafür zu nutzen.



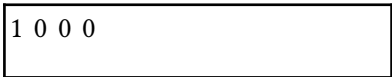
0

⋮



1

⋮



1 0 0 0

Ping Pong

Schreiben Sie ein Programm, welches beide Displayzeilen komplett mit Nullen füllt. Nun soll auf der oberen Zeile flüssig eine 1 von links nach rechts durch die Anzeige des Displays geschoben werden. Ist die 1 auf der rechten Seite angekommen, soll diese sich auf der unteren Zeile in die andere Richtung bewegen. Machen Sie die Geschwindigkeit mittels einer Definition anpassbar und lassen Sie die 1 in Dauerschleife von Seite zu Seite und zurück wandern. Nutzen Sie weiterhin **keine** vorgefertigten Funktionen.

1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

⋮

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

Binary Calculator II

Modifizieren Sie Ihr Programm aus der vorherigen Übung, sodass Sie künftig als Ausgabemedium für den Binary Calculator das LCD verwenden.

- Nutzen Sie für Ihre Rechenoperationen eine 32-Bit Zahl, lassen Sie die Zahl im **Dezimalsystem** ausgeben.
- Lassen Sie Operationen mit **negativem** Wert zu.
- Nutzen Sie aus Optimierungsgründen weiterhin **Bitoperation** für die Division/Multiplikation durch/mit der Zahl 2, sowie wenn nötig für das Tauschen des Vorzeichens der aktuellen Zahl (Zweierkomplement).
- Nutzen Sie weiterhin **keine** vorgefertigten Funktionen.

Beachten Sie alle Grenzfälle, die bei Verwendung negativer Zahlen im Zweierkomplement auftreten können!

Literaturverzeichnis

[1] Adafruit Industries, „Adafruit Fritzing Library“.